

03

Módulo 03 - Técnicas de detecção de objetos baseadas em Deep Learning

Visão de Computador e Stable Diffusion

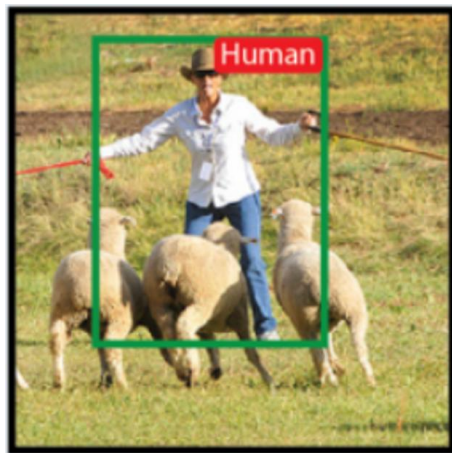
- ◆ **Deteção de Objetos:** Técnicas com Deep Learning (R-CNN, YOLO, SSD, Mediapipe), desempenho e uso de modelos pré-treinados.
- ◆ **Rastreamento em Tempo Real:** Diferenças face à deteção, algoritmos de DL e uso de OpenCV.
- ◆ **YOLO:** História, arquitetura, evolução (v1 a v13/v26) e comparação com outros métodos.
- ◆ **Aplicação do YOLO:** Configuração, modelos pré-treinados, treino, deteção em imagens/vídeo.
- ◆ **Otimização:** Ajustes, métricas de avaliação e melhorias contínuas em precisão e velocidade.
- ◆ Lab

Tarefas comuns em imagem/vídeo

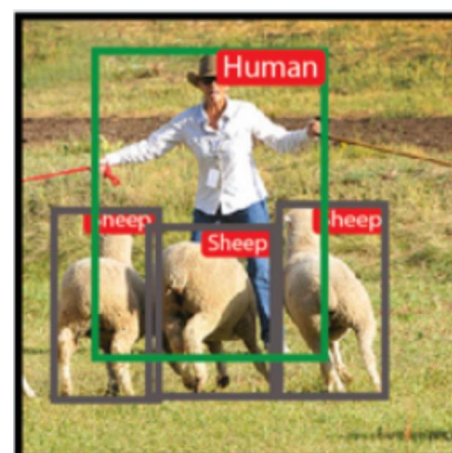
Imagem Original



Localização de Objetos
Object Localization



Reconhecimento de Objetos
Object Recognition



Segmentação Semântica
Semantic Segmentation



Segmentação por Instâncias
Instance Segmentation



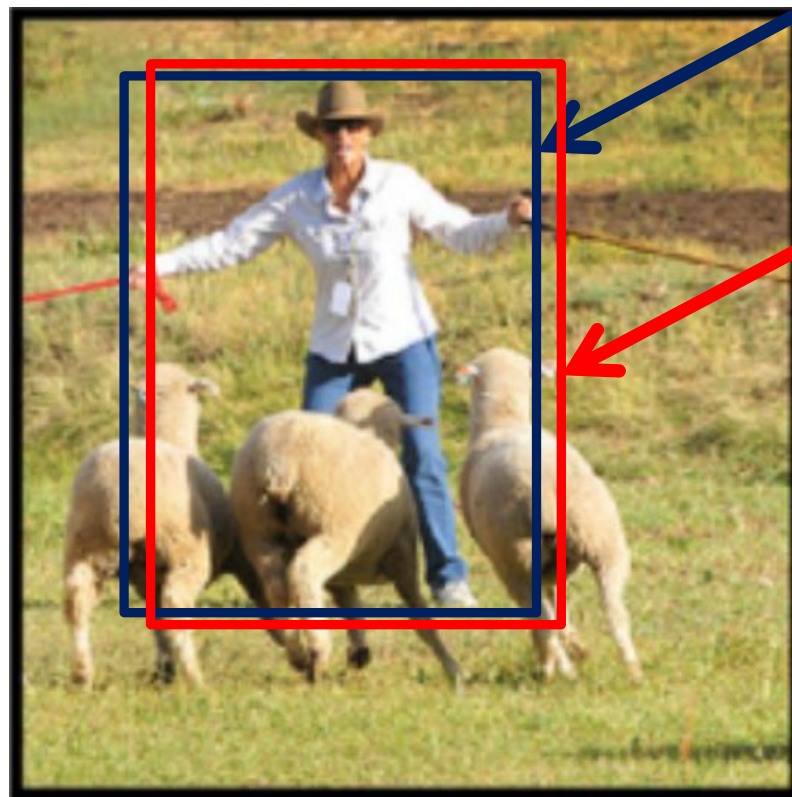
Deteção de Pontos-Chave
Keypoint Detection



Rotular cada pixel de acordo com a classe de objeto

Proposta de caixa delimitadora

Region of interest, region proposal, box proposal



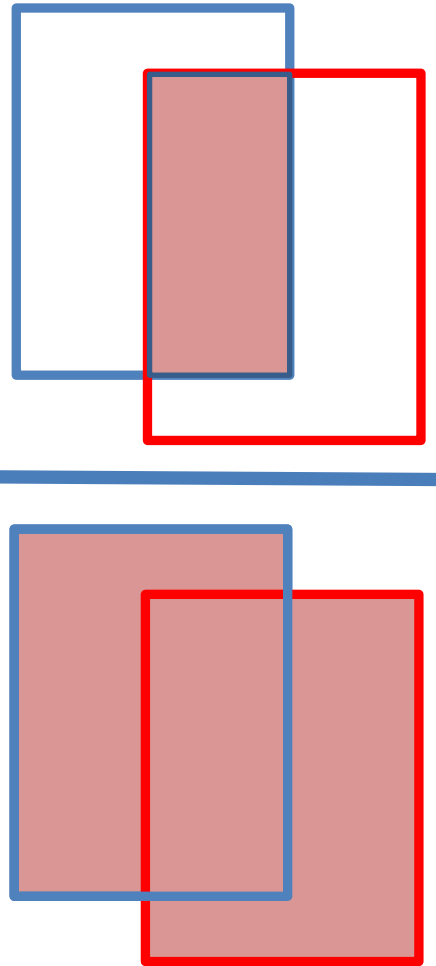
Ground truth
(Verdade fundamental)

Proposed bounding box
(Caixa delimitadora proposta)

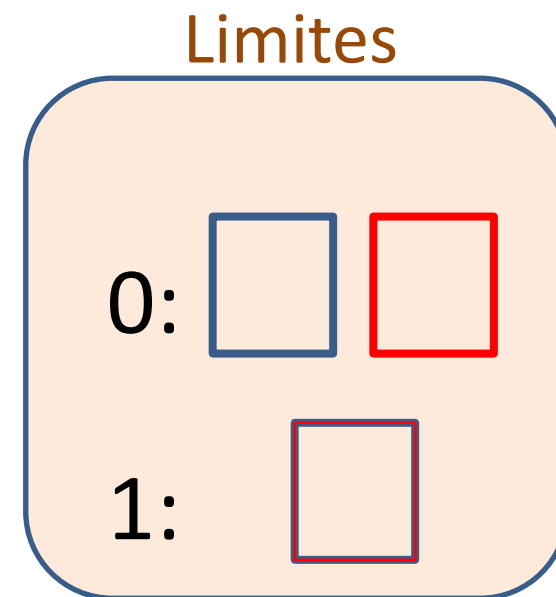
5 Parâmetros

- w, h
- x, y
- confidence score
índice de confiança : a probabilidade de conter um objeto e a precisão da caixa

O quanto correto: Intersection over Union (IOU)

$$\text{IOU} = \frac{\text{Área de sobreposição}}{\text{Área da União}}$$


The diagram illustrates the IOU formula with two examples of overlapping rectangles. In the top example, a blue rectangle and a red rectangle overlap, with the intersection area shaded in light red. In the bottom example, a blue rectangle and a red rectangle overlap, with the intersection area shaded in light red. A horizontal blue line separates the two examples.



O indicador *mAP50-95* é usado para saber **quão bem o modelo encontra objetos** - ele conta acertos com **IoU de 0,50 até 0,95**, e faz a média.

mAP50-95 (oficial COCO – Dataset “Common Objects in Context”)

1. Mede a qualidade de um detetor de objetos com vários níveis de exigência de **IoU**.
2. Calcula o **mAP** em **10 limiares fixos** de IoU: 0,50; 0,55; ...; 0,95
3. Depois faz a média desses 10 valores

$$\text{mAP}_{50-95} = \frac{\text{mAP}_{0.50} + \dots + \text{mAP}_{0.95}}{10}$$

Se uma deteção tem **IoU** = **0,82**:

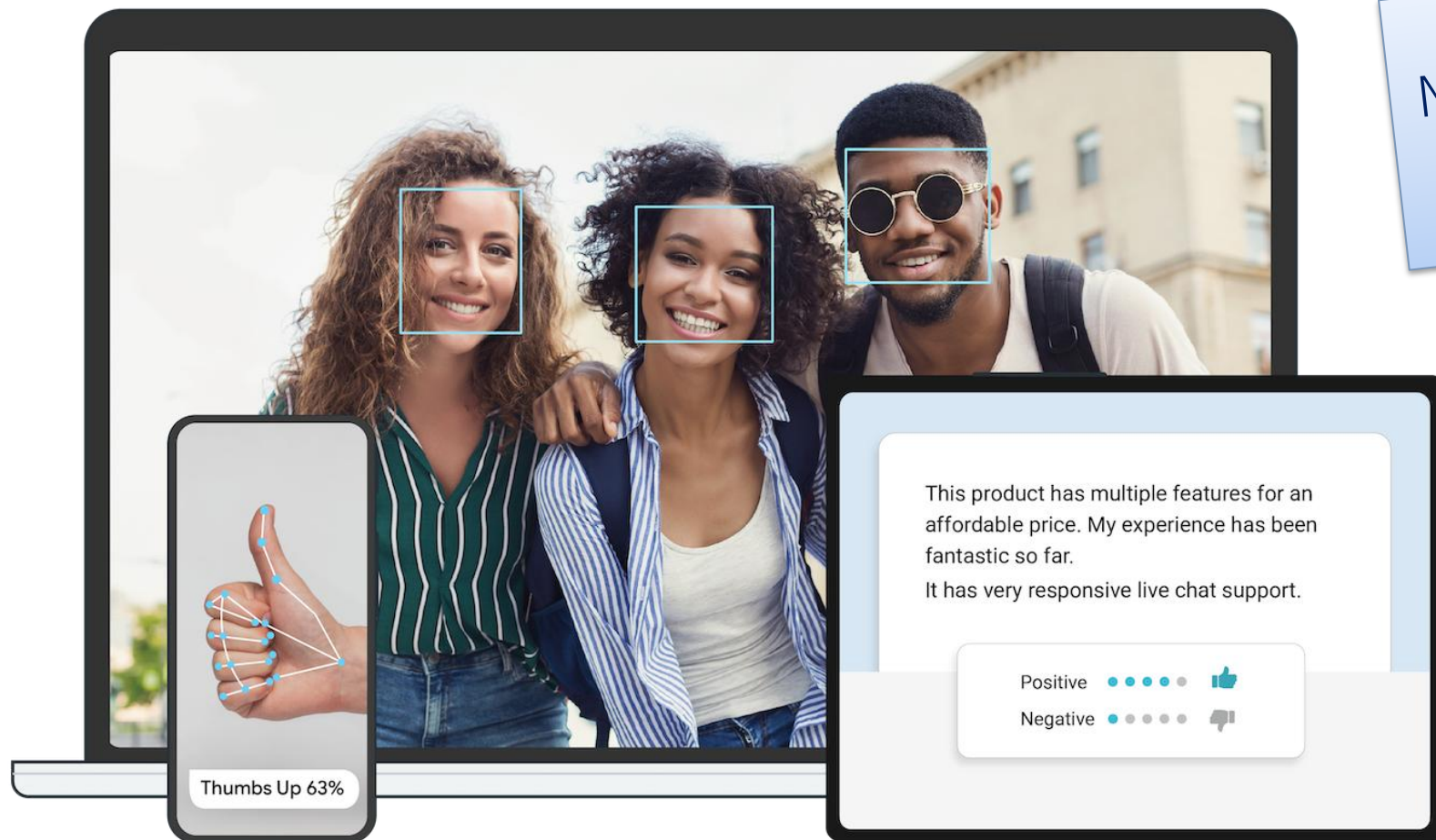
- No degrau **0,80 ela entra**, porque $0,82 \geq 0,80$.
- No degrau **0,85 ela já não entra**, porque $0,82 < 0,85$.

Mediapipe

Desde 2019
(v0.5)

Última v0.10.35
(a 2026-04-28)

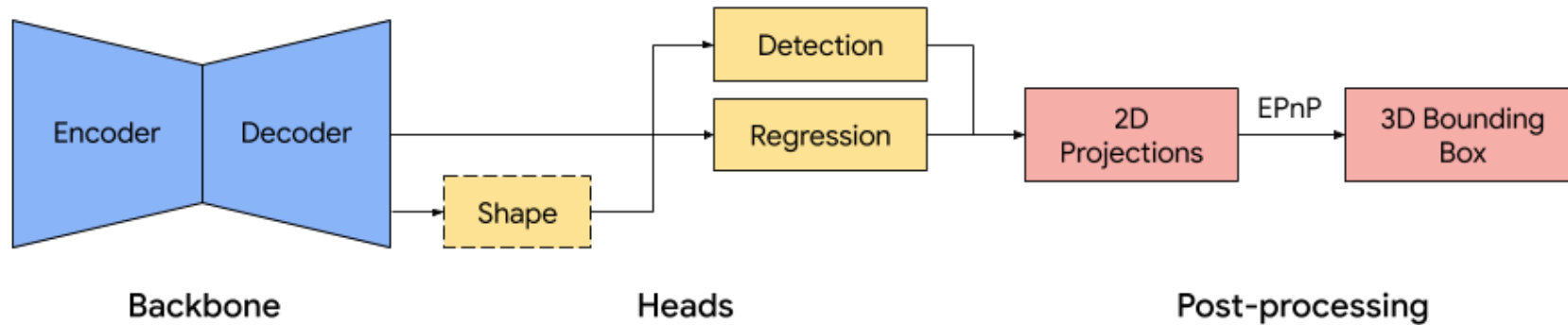
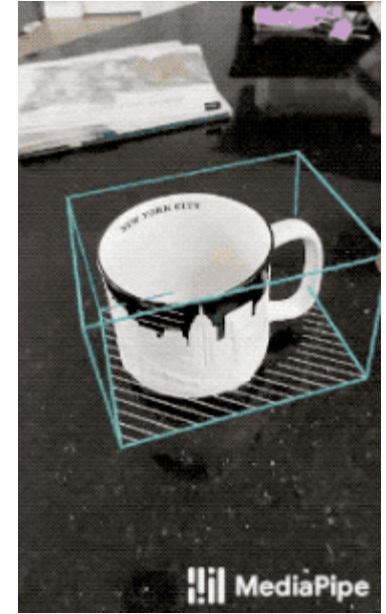
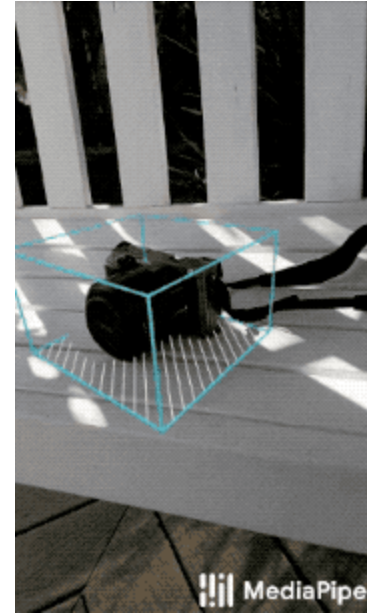
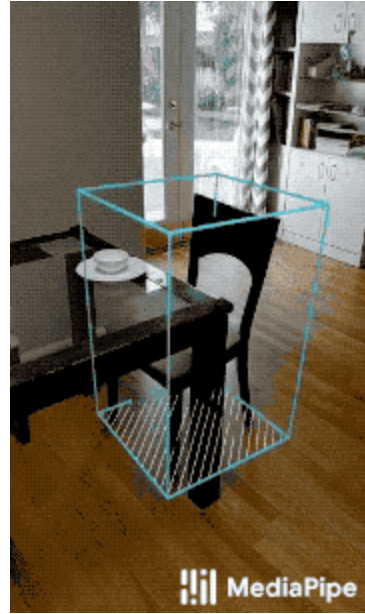
Mas só python
3.12!



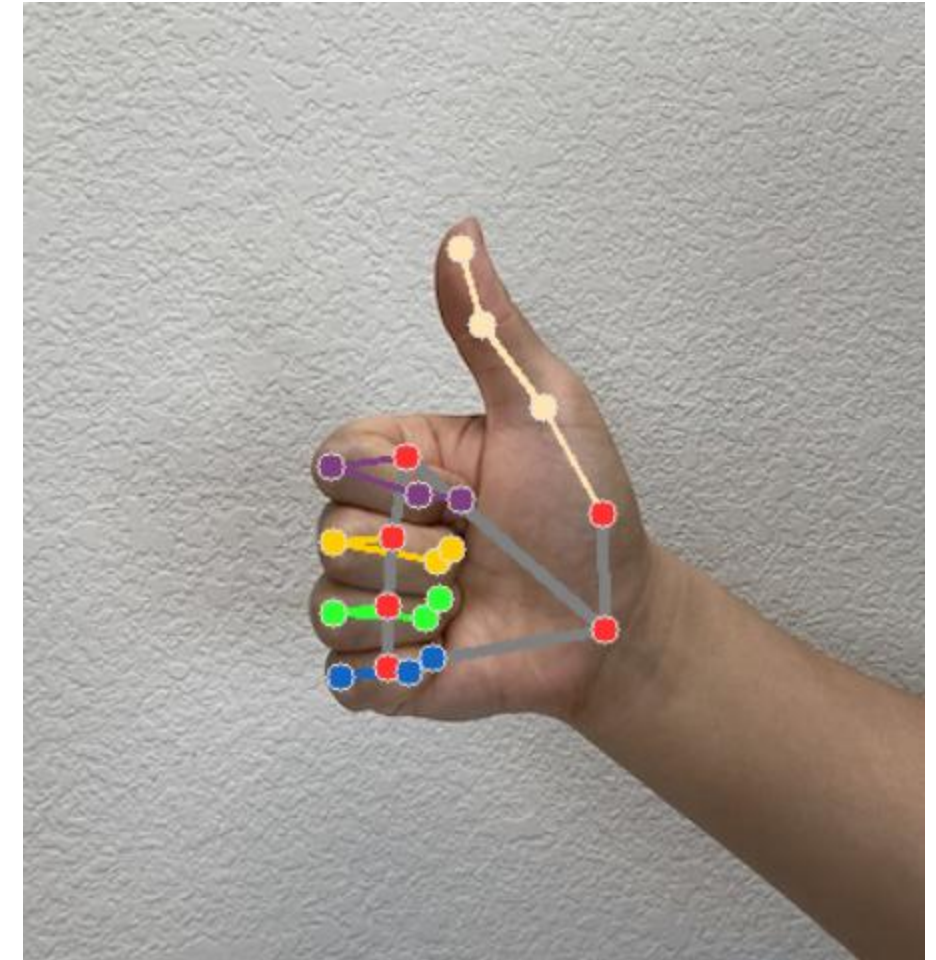
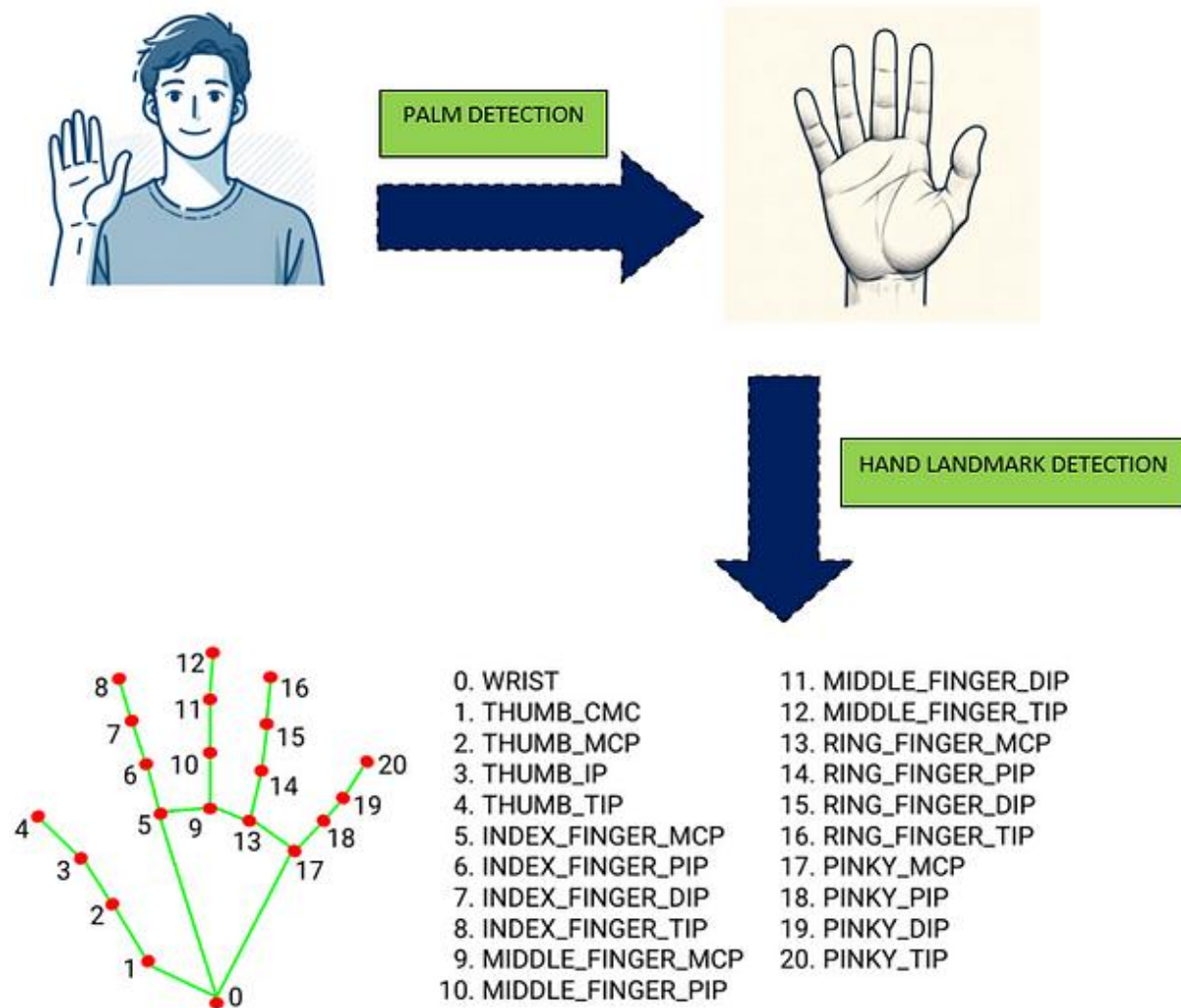
◆ Plataforma da **Google** para criar pipelines multimodais de ML (visão, áudio)

<https://ai.google.dev/edge/mediapipe/solutions/guide>

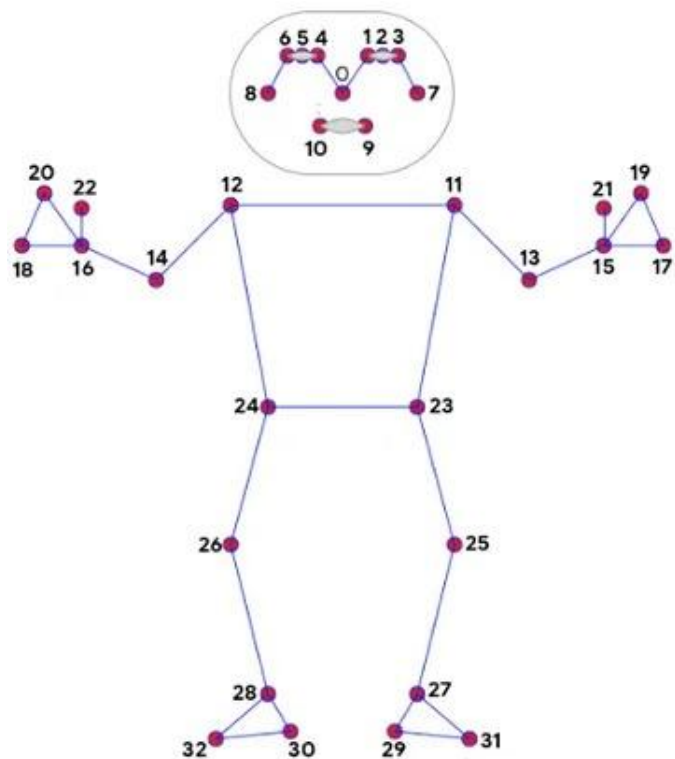
Mediapipe – rastrear objetos



Mediapipe - rastrear mão (hand tracking)



Mediapipe - rastrear corpo (pose tracking)

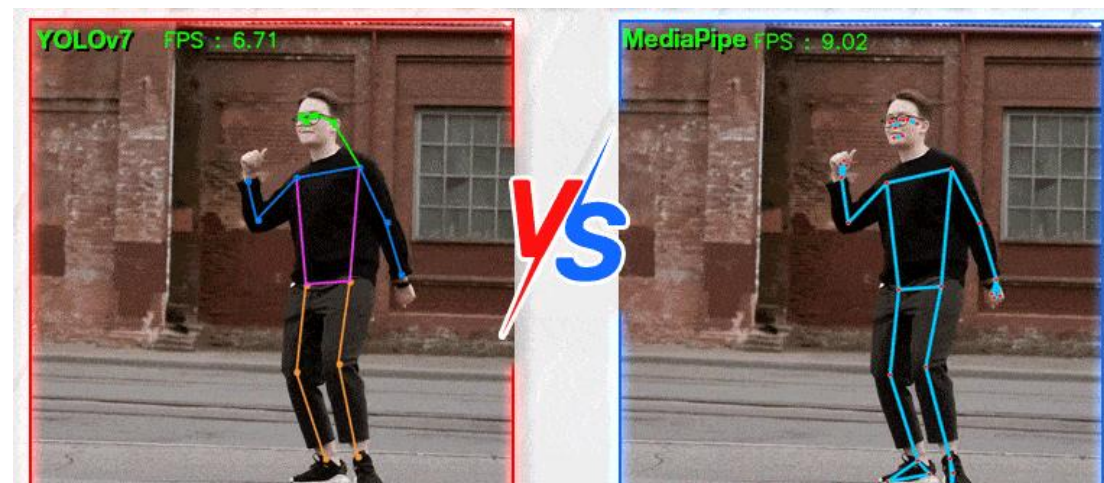


- 0. nose
- 1. left_eye_inner
- 2. left_eye
- 3. left_eye_outer
- 4. right_eye_inner
- 5. right_eye
- 6. right_eye_outer
- 7. left_ear
- 8. right_ear
- 9. mouth_left
- 10. mouth_right
- 11. left_shoulder
- 12. right_shoulder
- 13. left_elbow
- 14. right_elbow
- 15. left_wrist
- 16. right_wrist
- 17. left_pinky
- 18. right_pinky
- 19. left_index
- 20. right_index
- 21. left_thumb
- 22. right_thumb
- 23. left_hip
- 24. right_hip
- 25. left_knee
- 26. right_knee
- 27. left_ankle
- 28. right_ankle
- 29. left_heel
- 30. right_heel
- 31. left_foot_index
- 32. right_foot_index



Frameworks e Modelos DeepLearning

	Frameworks	Modelos
Exemplos	MediaPipe, OpenPose, DeepStream	YOLO, SSD, R-CNN
Tipo	Conjunto de pipelines com modelos integrados	Modelos de deep learning isolados
Necessidade de treino	Normalmente não — usam modelos pré-treinados	Sim — requer treino ou fine-tuning
Foco	Pose, rastreamento, rosto, mãos, vídeo em tempo real	Deteção, segmentação e classificação genérica
Facilidade de uso	Alta — prontos para integrar em apps e mobile	Média — exigem configuração e treino
Desempenho	Otimizados para CPU e dispositivos edge	Melhor em GPU, com maior precisão e flexibilidade



Ex 01: Detecção de Objetos 3D em Tempo Real com MediaPipe Objectron

Ex 02: Detecção e Acompanhamento de Mãos com MediaPipe Hands

Histórico modelos de detecção de objetos

R-CNN → **OverFeat** → MultiBox → SPP-Net → MR-CNN → DeepBox → AttentionNet →
2013.11 ICLR' 14 CVPR' 14 ECCV' 14 ICCV' 15 ICCV' 15 ICCV' 15

Fast R-CNN → DeepProposal → **Faster R-CNN** → **OHEM** → **YOLO v1** → G-CNN → AZNet →
ICCV' 15 ICCV' 15 NIPS' 15 CVPR' 16 CVPR' 16 CVPR' 16 CVPR' 16

Inside-OutsideNet(ION) → HyperNet → CRAFT → MultiPathNet(MPN) → **SSD** → GBDNet →
CVPR' 16 CVPR' 16 CVPR' 16 BMVC' 16 ECCV' 16 ECCV' 16

CPF → MS-CNN → **R-FCN** → PVANET → DeepID-Net → NoC → DSSD → TDM → **YOLO v2** →
ECCV' 16 ECCV' 16 NIPS' 16 NIPS' 16 PAMI' 16 TPAMI' 16 arXiv' 17 CVPR' 17 CVPR' 17

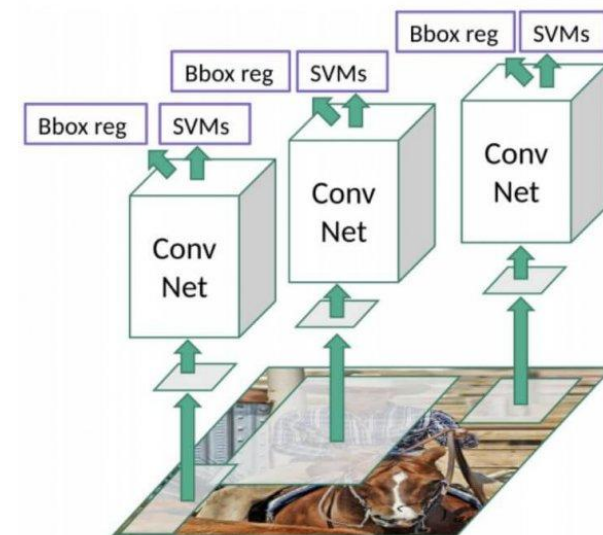
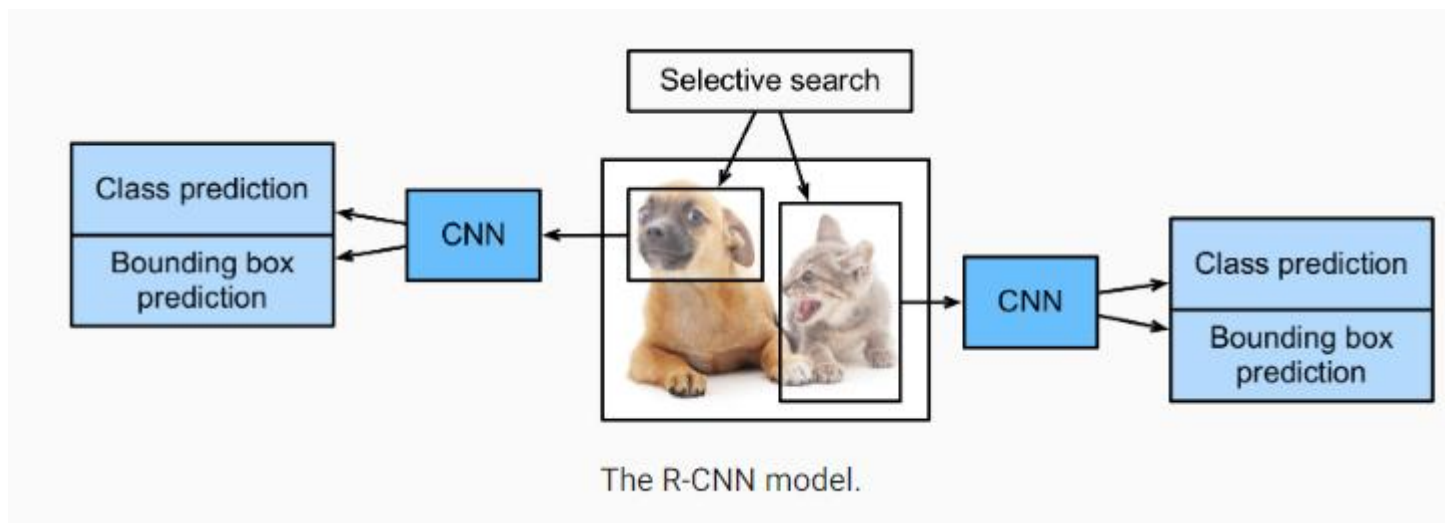
Feature Pyramid Net(**FPN**) → RON → DCN → DeNet → CoupleNet → **RetinaNet** → DSOD →
CVPR' 17 CVPR' 17 ICCV' 17 ICCV' 17 ICCV' 17 ICCV' 17 ICCV' 17

Mask R-CNN → SMN → **YOLO v3** → SIN → STDN → **RefineDet** → MLKP → Relation-Net →
ICCV' 17 ICCV' 17 arXiv' 18 CVPR' 18 CVPR' 18 CVPR' 18 CVPR' 18 CVPR' 18

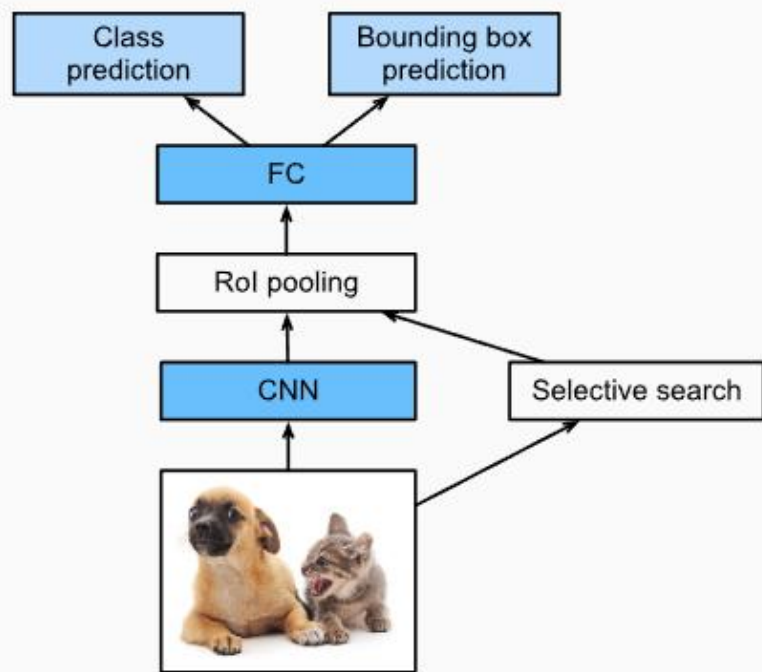
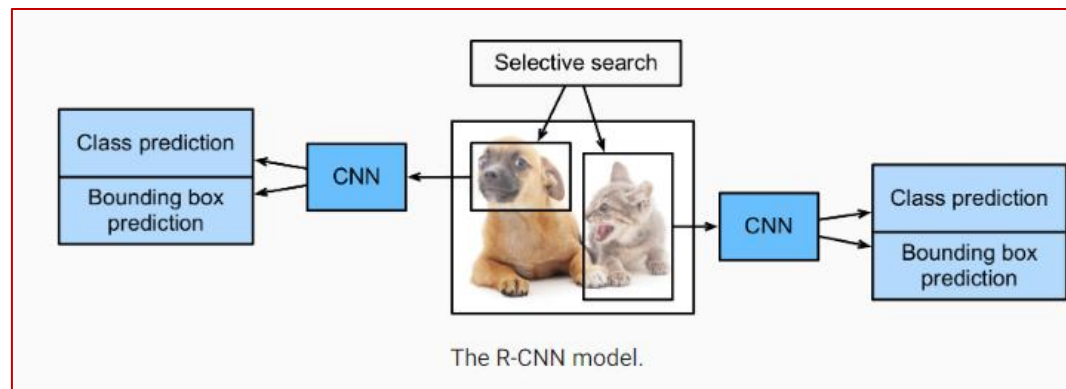
Cascade R-CNN → RFBNet → CornetNet → Pelee → MethAnchor → SNIPER → **M2Det** ...
CVPR' 18 ECCV' 18 ECCV' 18 NIPS' 18 NIPS' 18 NIPS' 18 AAAI' 19

Region-based CNN (R-CNN)

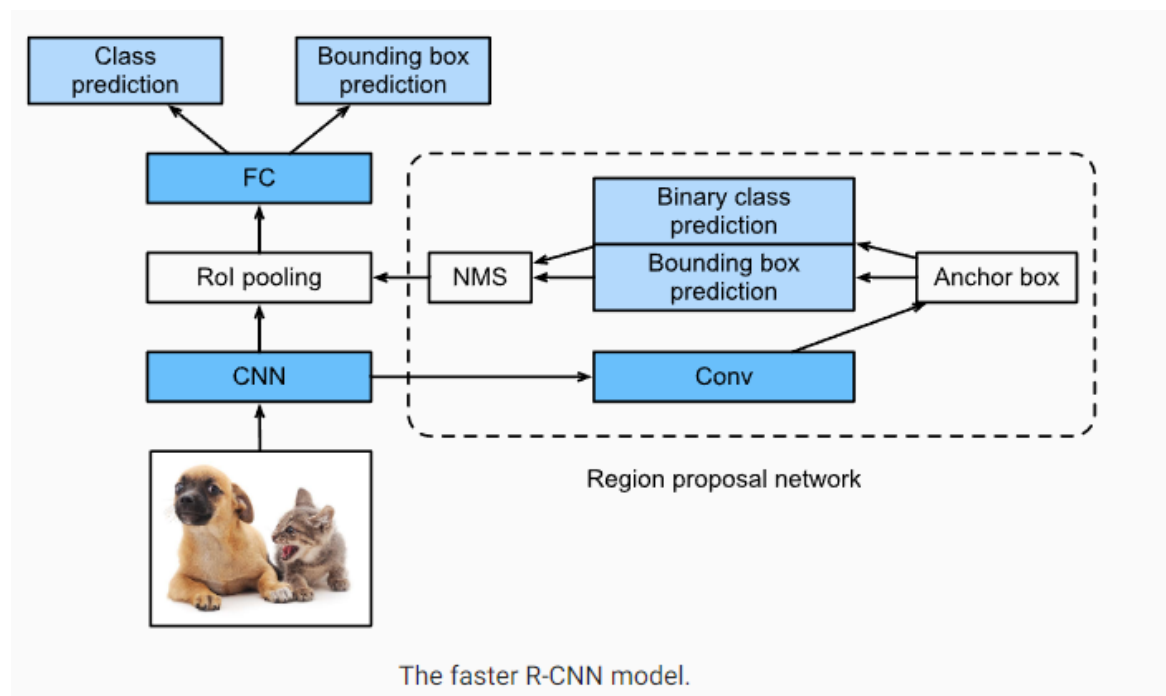
A **R-CNN** extrai primeiro muitas (por exemplo, 2000) propostas de regiões da imagem de entrada rotulando as suas classes e caixas delimitadoras (por exemplo, deslocamentos).



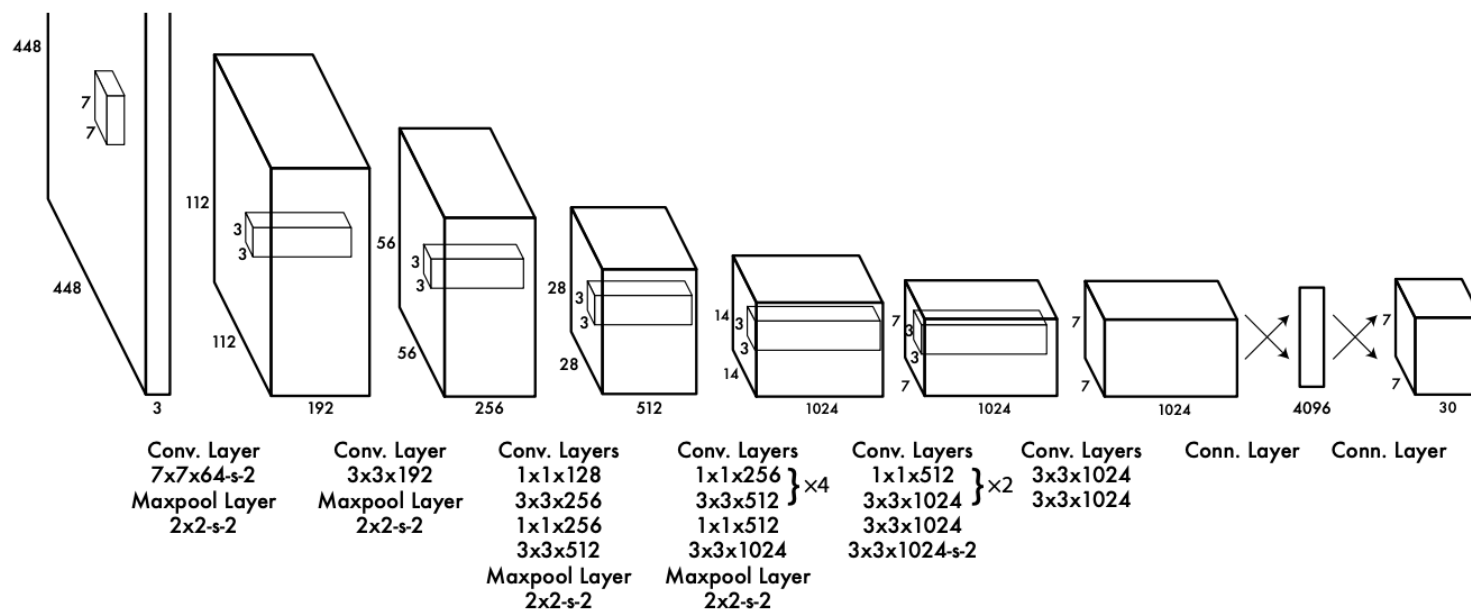
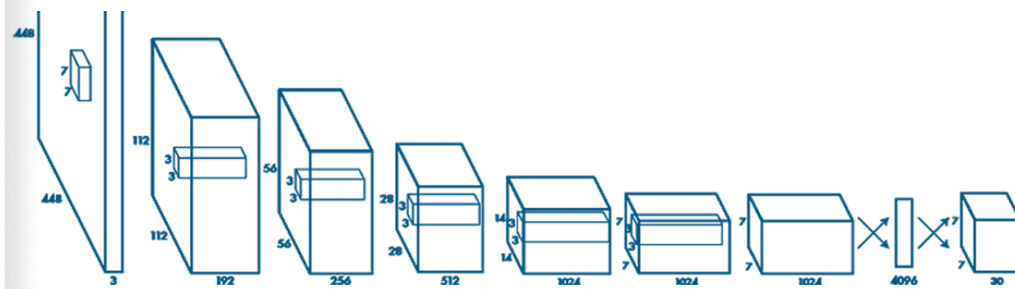
Fast R-CNN e Faster R-CNN (melhores modelos)



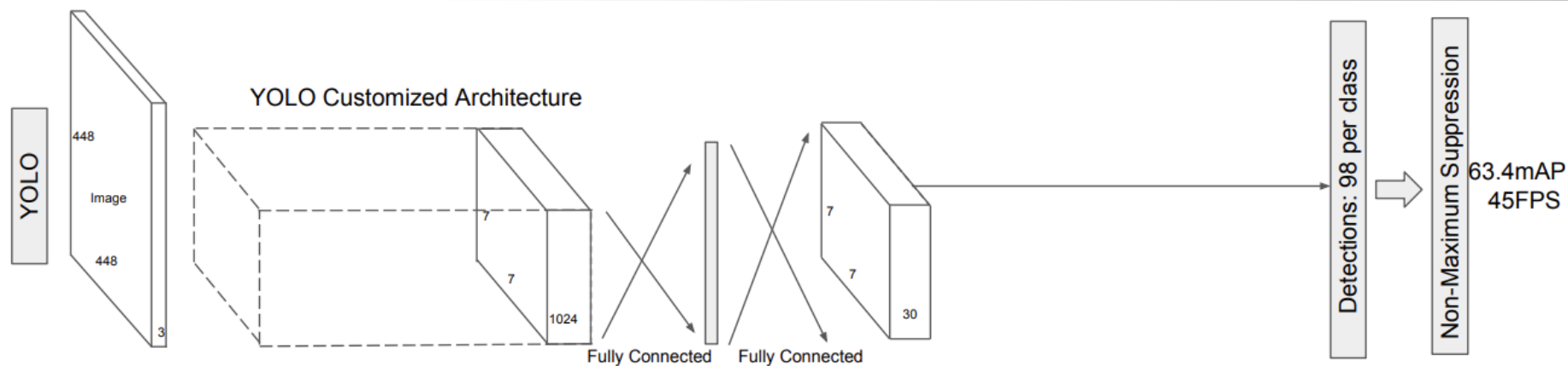
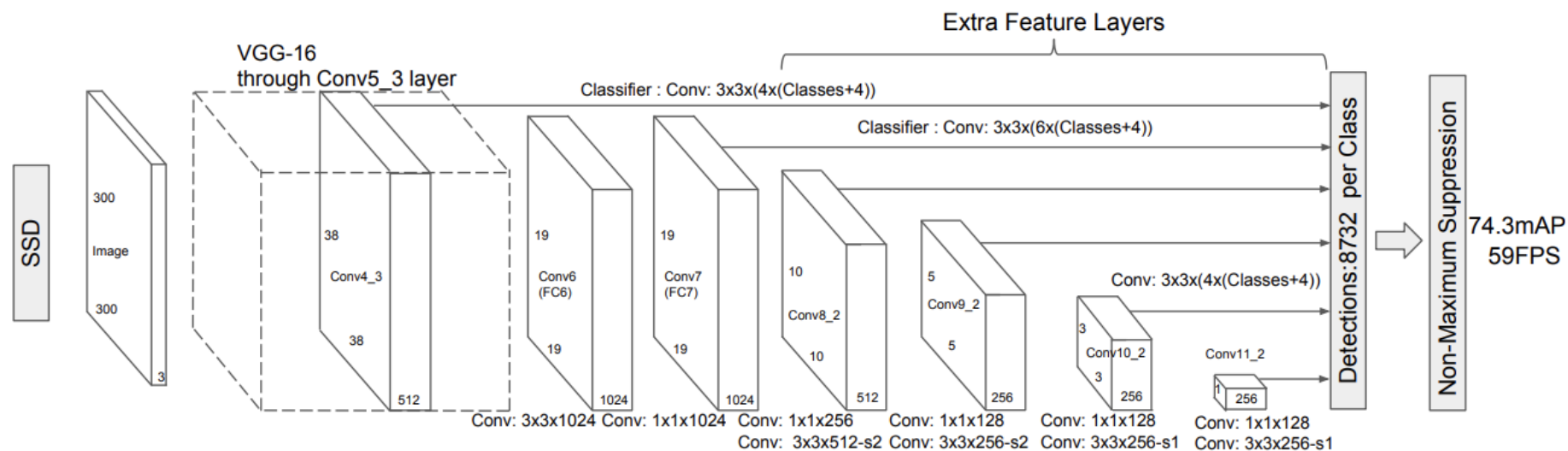
The fast R-CNN model.



YOLO (You Only Look Once)



SSD (Single Shot MultiBox Detector) vs YOLO



Resumo das arquiteturas populares de detecção de objetos

- ◆ R-CNN

- ◆ Fast R-CNN

- ◆ Faster R-CNN

→ **emprega uma Rede de Propostas de Região (RPN)** para identificar propostas de objetos, **seguida por uma rede R-CNN** para classificar e localizar objetos

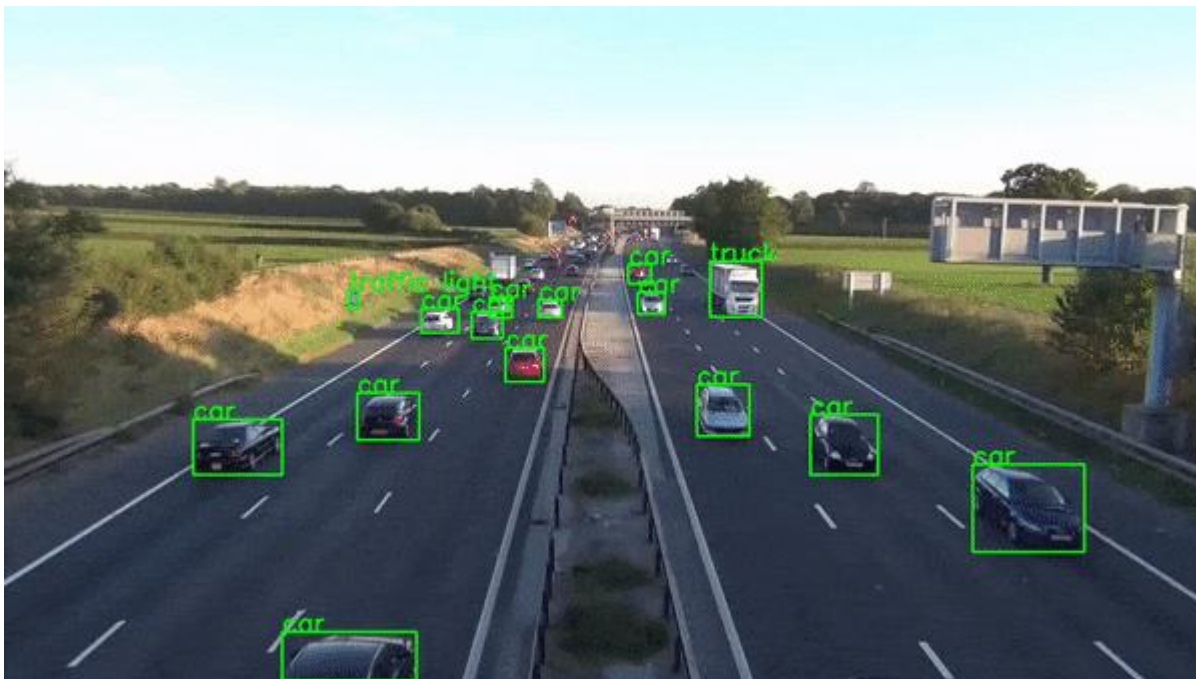
(Two-Stage/Proposal)

- ◆ YOLO

- ◆ SSD

→ **uma única CNN** para detectar objetos e prever suas caixas delimitadoras

YOLO (You Only Look Once)



Criado em 2015 por **Joseph Redmon**

Mas ele **parou a pesquisa CV** devido a preocupações éticas!

Joseph Redmon · Feb 20, 2020
@pjreddie · [Follow](#)
"We shouldn't have to think about the societal impact of our work because it's hard and other people can do it for us" is a really bad argument.

Roger Grosse @RogerGrosse
Replying to @kevin_zakka and @hardmaru
To be clear, I don't think this is a positive step. Societal impacts of AI is a tough field, and there are researchers and organizations that study it professionally. Most authors do not have expertise in the area and won't do good enough scholarship to say something meaningful.

Joseph Redmon
@pjreddie · [Follow](#)
I stopped doing CV research because I saw the impact my work was having. I loved the work but the military applications and privacy concerns eventually became impossible to ignore.

Roger Grosse @RogerGrosse
Replying to @skoularidou
What's an example of a situation where you think someone should decide not to submit their paper due to Broader Impacts reasons?

4:09 PM · Feb 20, 2020

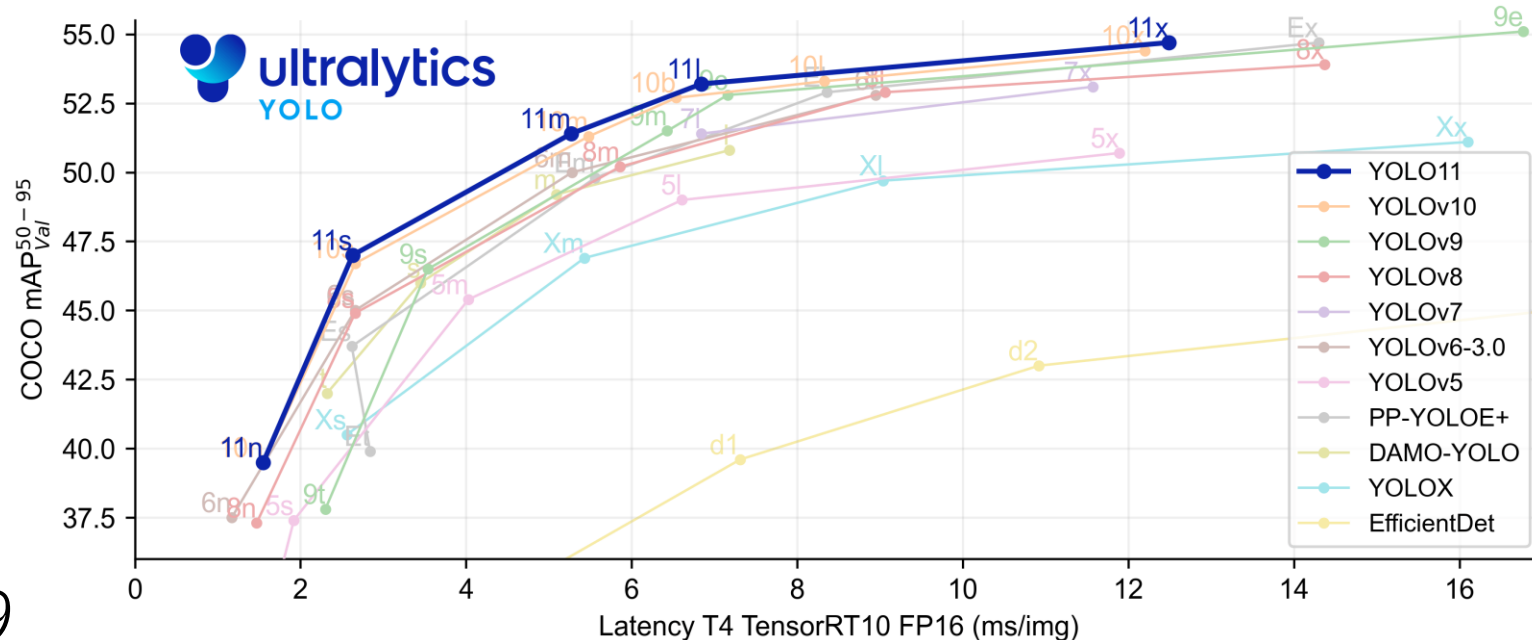
3.3K Reply Share

[Read 77 replies](#)

YOLO (You Only Look Once)

Várias versões (YOLOv1 até YOLOv13 e Yolov26) com melhorias constantes em precisão e velocidade.

- ◆ <https://www.ultralytics.com/yolo>
- ◆ <https://github.com/ultralytics/ultralytics>



Uma Licença MIT de YOLOv9

- ◆ <https://github.com/WongKinYiu/YOLO>

Versões e escalas YOLO

```
from ultralytics import YOLO

# VALIDAÇÃO =====
model = YOLO('yolov13{n/s/l/x}.pt') # Substituir pela escala de modelo desejada

# TREINO =====
model = YOLO('yolov13n.yaml')

# Treinar o modelo
results = model.train(
    data='coco.yaml',
    epochs=600,
    batch=256,
    imgsz=640,
    scale=0.5, # S:0.9; L:0.9; X:0.9
    mosaic=1.0,
    mixup=0.0, # S:0.05; L:0.15; X:0.2
    copy_paste=0.1, # S:0.15; L:0.5; X:0.6
    device="0,1,2,3",
)

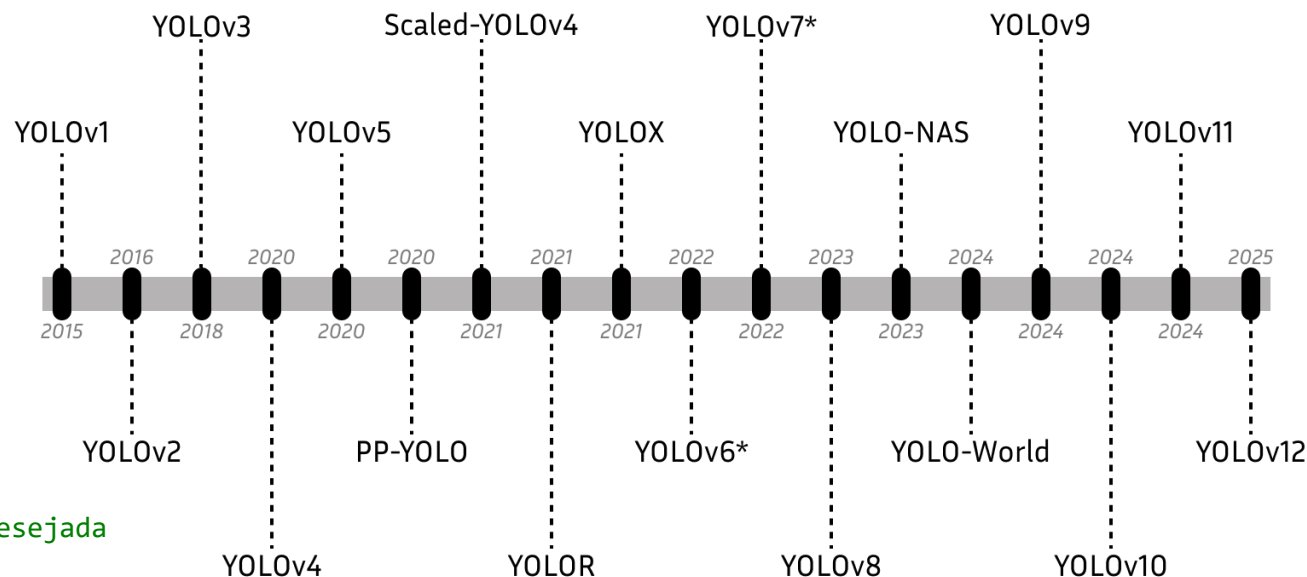
# Avaliar o desempenho do modelo no conjunto de validação
metrics = model.val('coco.yaml')

# Realizar detecção de objetos numa imagem
results = model("path/to/your/image.jpg")
results[0].show()

# PREDIÇÃO =====
model = YOLO('yolov13{n/s/l/x}.pt') # Substituir pela escala de modelo desejada
model.predict()

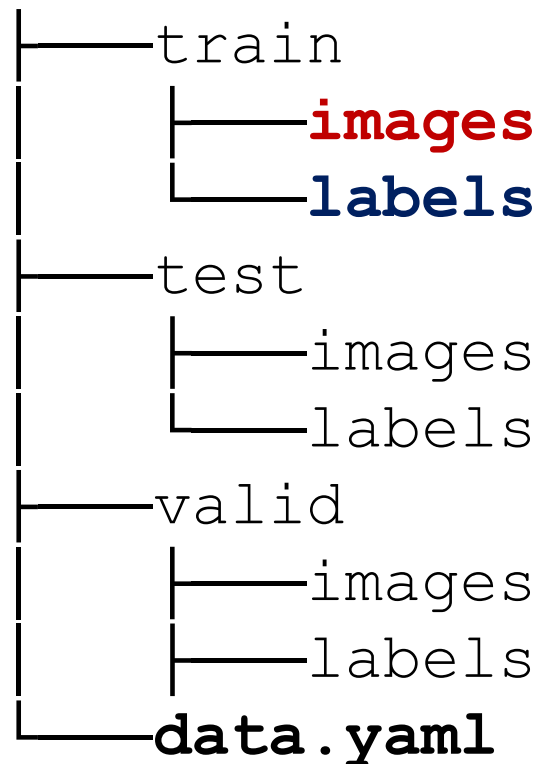
# EXPORTAÇÃO =====
model = YOLO('yolov13{n/s/l/x}.pt') # Substituir pela escala de modelo desejada
model.export(format="engine", half=True) # ou format="onnx"
```

n (Nano): Muito leve e rápida, menor precisão.
s (Small): Equilíbrio entre rapidez e precisão.
l (Large): Alta precisão, requer mais GPU.
x (Extra-Large): Máxima precisão, mais lenta.



Preparar dados para treinar novo modelo YOLOv11

Dataset



a1.jpg
a2.jpg
...

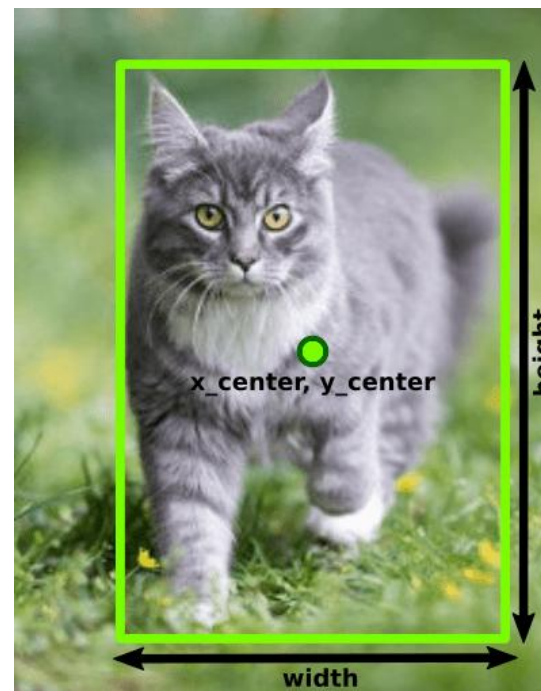
a1.txt
a2.txt
...

{object_class_id} {x_center} {y_center} {width} {height}

$x_center = (box_x_left + box_x_width / 2) / image_width$
 $y_center = (box_y_top + box_height / 2) / image_height$
 $width = box_width / image_width$
 $height = box_height / image_height$

```
train: ../train/images  
val: ../val/images  
  
nc: 2  
names: ['cat', 'dog']
```

0 1 ,3,4,5,...



Treinar o novo modelo YOLOv11



Image:

image_width = 612
image_height = 415

train/images/cat_dog.jpg

data.yaml

train: ../train/images
val: ../val/images

nc: 2
names: ['cat','dog']

Objetos:

Dog	Cat
box_x_left=261	box_x_left=140
box_x_top=94	box_x_top=170
box_width=200	box_width=116
box_height=219	box_height=146

Dog (class id=1):

$x_center = (261+200/2)/612 = 0.589869281$
 $y_center = (94+219/2)/415 = 0.490361446$
 $width = 200/612 = 0.326797386$
 $height = 219/415 = 0.527710843$

Cat (class id=0)

$x_center = (140+116/2)/612 = 0.323529412$
 $y_center = (170+146/2)/415 = 0.585542169$
 $width = 116/612 = 0.189542484$
 $height = 146/415 = 0.351807229$

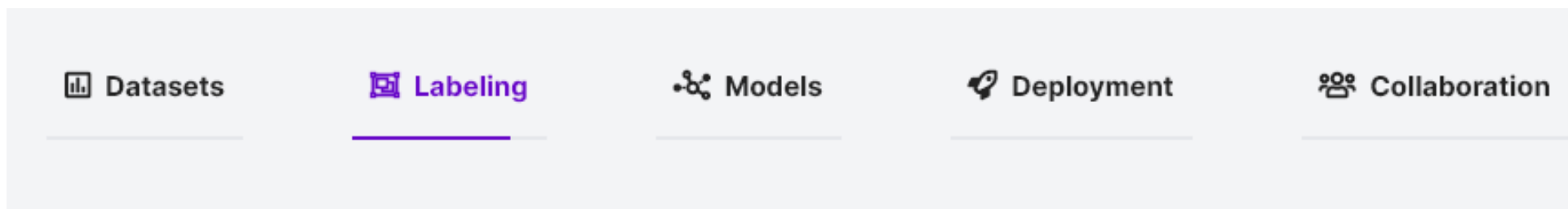
train/labels/cat_dog.txt

1 0.589869281 0.490361446 0.326797386 0.527710843
0 0.323529412 0.585542169 0.189542484 0.351807229

- ◆ <https://roboflow.com/>

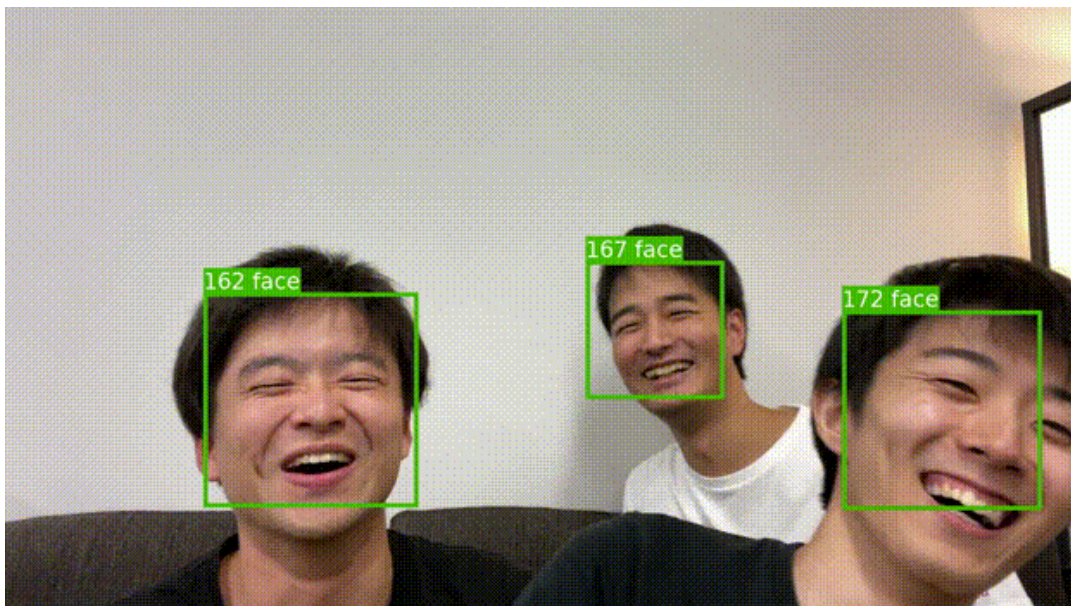


Simplifica a construção e implantação de modelos de visão computacional



- ◆ <https://www.youtube.com/watch?v=a3SBRtILjPI&t=339s>

Controlar a perda do objeto



- StrongSort

<https://github.com/dyhBUPT/StrongSORT>

- ByteTrack

<https://github.com/ifzhang/ByteTrack>

- NorFair

<https://github.com/tryolabs/norfair>

- DeepSORT

https://github.com/nwojke/deep_sort

- Sort, OcSort, ...

Ex 03: Detecção de objetos com YOLOv11

Ex 04: Segmentação por Instâncias com YOLOv11